

CTS SRE Day 15, 16

Incident Management and Fault Tolerance 1 & 2

EMPOWERING High Performance
Technology Teams

Agenda for the Day

1. Introduction to Incident Management & Fault Tolerance
2. Incident Lifecycle Automation
3. Root Cause Analysis (RCA) with Monitoring Tools
4. Fault-Tolerant Architecture Design
5. Redundancy & Failover Testing
6. Scalability & Recovery Mechanisms
7. Fault Isolation & Containment
8. Graceful Degradation Strategies
9. Proactive Monitoring & Alerting
10. Fault Injection & Chaos Engineering
11. Blameless Postmortem & Continuous Improvement

Introduction to Incident Management & Fault Tolerance

Objective:

- Ensure systems can **detect, respond, recover, and learn from failures.**
- Implement **automated response mechanisms** to minimize downtime.
- Design **highly available and fault-tolerant** architectures.

Key Components:

- ✓ Incident Lifecycle Management
- ✓ Monitoring & RCA
- ✓ Auto-healing & Fault Tolerance
- ✓ Redundancy & Failover
- ✓ Chaos Engineering
- ✓ Blameless Postmortems

Incident Lifecycle Automation

Objective: Automate the full incident lifecycle from detection to resolution.

Key Steps:

- ✓ **Incident Detection:** Use Prometheus Alert Rules, AWS CloudWatch Alarms
- ✓ **Incident Escalation:** Integrate with PagerDuty, SNS, or Lambda Functions
- ✓ **Incident Resolution:** Implement automated rollback & self-healing scripts
- ✓ **Postmortem Documentation:** Capture root cause, impact, and improvements

Tools Used: Prometheus, CloudWatch, PagerDuty, AWS Lambda

Root Cause Analysis (RCA) with Monitoring Tools



Objective: Identify, analyze, and resolve the root cause of incidents.

Steps for RCA:

- **Simulate incidents** (e.g., CPU spikes, memory leaks, network throttling)
- **Use Prometheus Metrics** for system performance analysis
- **Build Grafana Dashboards** to visualize errors & performance trends
- **Correlate logs & traces** from AWS CloudWatch & AWS X-Ray
- **Document Findings** & recommend preventive measures

Tools Used: Prometheus, Grafana, CloudWatch, X-Ray

Fault-Tolerant Architecture Design

Objective: Ensure high availability & minimal downtime.

Core Principles:

- ✓ **Scalability** – Use Auto Scaling for dynamic capacity management
- ✓ **Load Balancing** – Distribute traffic efficiently across instances
- ✓ **Multi-Region Redundancy** – Implement Route 53 DNS-based failover
- ✓ **Resilient Database** – Use Amazon RDS Multi-AZ for failover support

Tools Used: Auto Scaling, ELB, Route 53, RDS Multi-AZ

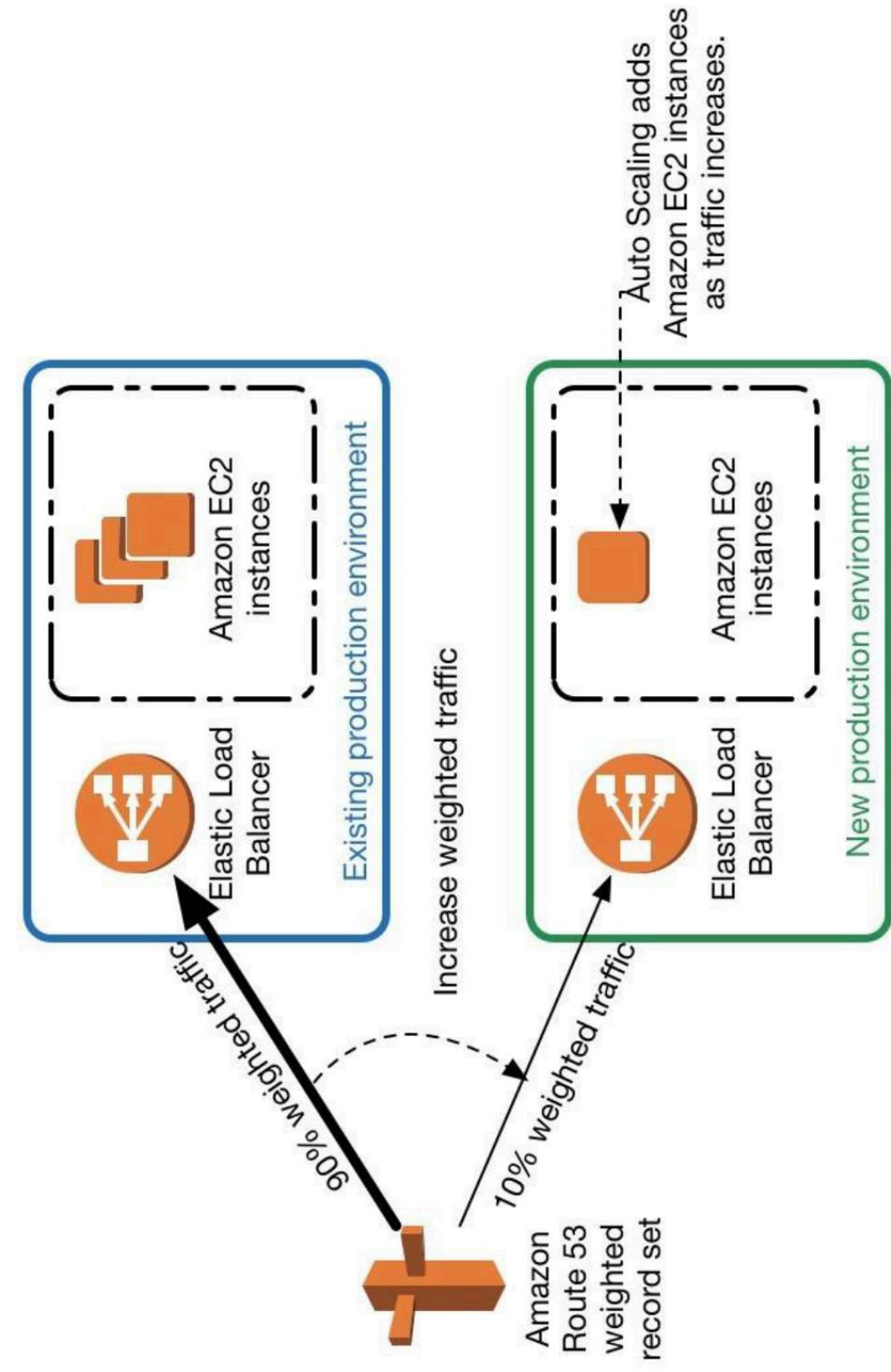
Redundancy & Failover Testing

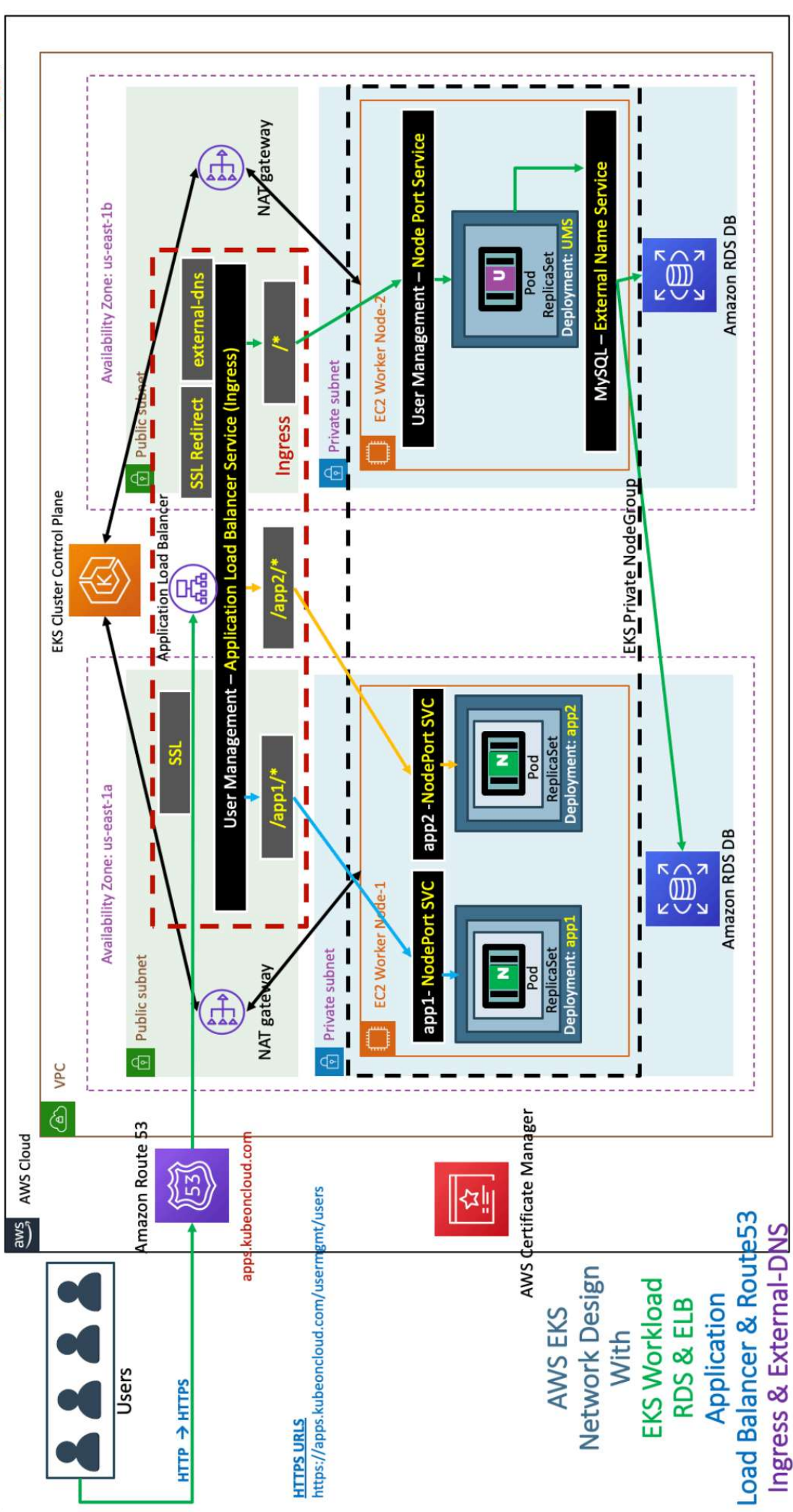
Objective: Validate resilience by simulating real-world failure scenarios.

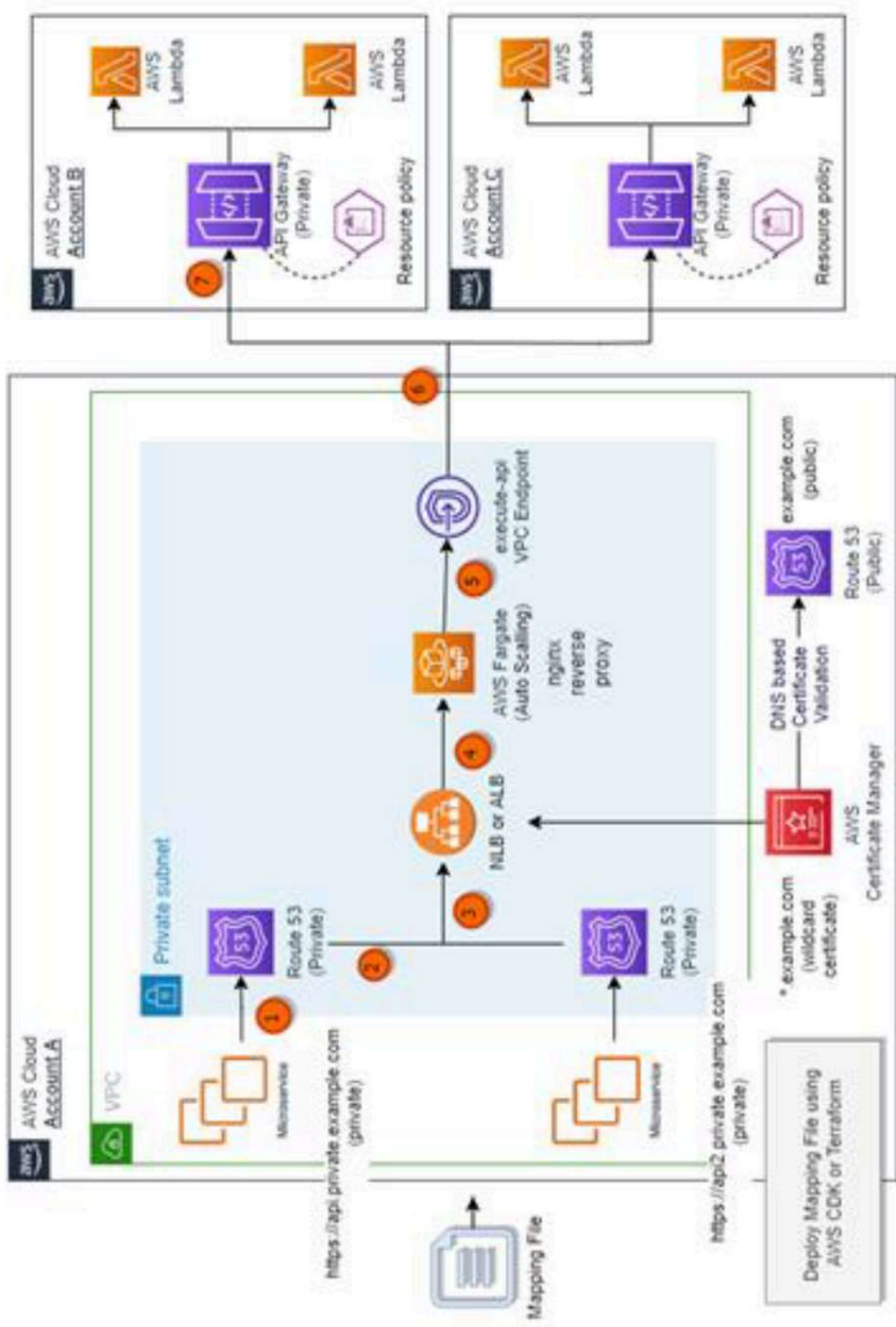
Key Strategies:

- ✓ **Active-Active vs Active-Passive Failover setups**
- ✓ **Terraform-based infrastructure provisioning**
- ✓ **Simulate network failures & measure failover speed**
- ✓ **Validate DNS failover & traffic redirection**

Tools Used: Terraform, Route 53, AWS Auto Scaling, RDS Multi-AZ







Scalability & Recovery Mechanisms

Objective: Ensure that systems can scale effectively and recover quickly.

Key Approaches:

- ✓ **Load Testing** – Validate auto-scaling triggers using JMeter
- ✓ **Automated Recovery** – Implement AWS Lambda self-healing scripts
- ✓ **Stress Testing** – Simulate peak loads to ensure system resilience

Tools Used: AWS Auto Scaling, JMeter, Lambda, Step Functions

Fault Isolation & Containment

Objective: Prevent cascading failures with circuit breakers & service mesh.

Key Methods:

- ✓ **Implement Istio Service Mesh** for microservices isolation
- ✓ **Deploy Circuit Breakers** to contain failures
- ✓ **Segment Resources** using AWS App Mesh & IAM Policies

Tools Used: Istio, AWS App Mesh, AWS IAM

Graceful Degradation Strategies

Objective: Maintain essential services during failures.

Key Approaches:

- ✓ **Feature Toggles** – Disable non-critical features during incidents
- ✓ **API Rate Limiting & Throttling** – Maintain fair usage
- ✓ **Degraded SLAs** – Reduce response quality while ensuring availability

Tools Used: API Gateway, AWS Lambda, Feature Flags

Proactive Monitoring & Alerting

Objective: Identify potential issues before they impact customers.

Key Strategies:

- ✓ **Anomaly Detection** – Use AI-based monitoring (Datadog, AWS AI)
- ✓ **SLI/SLO-based Alerts** – Define & measure key system health indicators
- ✓ **Synthetic Monitoring** – Simulate user traffic & detect issues

Tools Used: AWS CloudWatch AI, Datadog, Prometheus, ELK Stack

Fault Injection & Chaos Engineering

Objective: Simulate failures to validate recovery strategies.

Key Approaches:

- ✓ **AWS Fault Injection Simulator (FIS)** – Simulate infrastructure failures
- ✓ **Chaos Monkey** – Randomly shut down instances
- ✓ **Validate Automated Rollback Mechanisms**

Tools Used: AWS FIS, Gremlin, Chaos Monkey

Blameless Postmortem & Continuous Improvement

Objective: Analyze incidents to drive long-term improvements.

Postmortem Process:

- ✓ **Capture Incident Timeline**
- ✓ **Analyze Root Causes & contributing factors**
- ✓ **Document Lessons Learned & prevention strategies**
- ✓ **Implement Infrastructure Improvements using Terraform**

Tools Used: CloudWatch Logs, Terraform, Postmortem Templates

Hands on Exercises

Labs

- [AWS EKS lab 1](#)
- [Incident Lifecycle Automation - Incident Management and Fault Tolerance .docx](#)
- [active-passive setups - Incident Management and Fault Tolerance .docx](#)
- [Security Incident Simulation - Incident Management and Fault Tolerance.docx](#)

Github link:

- <https://github.com/AshishSharmaPrivate/Lab/blob/main/active-passive%20setups%20Incident%20Management%20and%20Fault%20Tolerance>
- <https://github.com/AshishSharmaPrivate/Lab/blob/main/Incident-automation-lifecycle.txt>
- <https://github.com/AshishSharmaPrivate/Lab/blob/main/Security%20Incident%20Simulation%20-%20Incident%20Management>

Summary & Key Takeaways

- ✓ **Automate Incident Response** using alerting & self-healing systems
- ✓ **Design Fault-Tolerant Architectures** with multi-region setups
- ✓ **Use Chaos Engineering** to proactively test system resilience
- ✓ **Adopt Blameless Postmortems** for continuous improvement
- ✓ **Ensure Proactive Monitoring** to detect failures before they happen

Thank You!
StackRoute
(An NIIT Venture)

